

Intermediate (swap in a structured parser) — Hints

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried. Each hint has 2 depth levels: **Basic** (the plain idea) and **Intermediate** (proper Python). Read Basic first even if you already know Python — it's the fastest way to spot exactly what Intermediate adds.

- [Hint 1 — The idea, and the exact pieces](#)
- [Hint 2 — The plan, and almost the whole thing](#)

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 1 — The idea, and the exact pieces

Basic Version

You're going to swap the last piece of the chain: instead of "just give me text back," you'll say "give me back an object that matches this exact shape."

To describe the shape, you write a small Pydantic class with named fields, like `name` and `age` . Then you tell your model, "always answer in this shape."

Once that's set up, try asking it something that clearly matches the shape, and something that clearly doesn't — and watch what happens differently.

- A Pydantic model: `from pydantic import BaseModel` then `class Person(BaseModel):`
`name: str; age: int`
- Attach it to the model: `structured_model =`
`ChatOpenAI().with_structured_output(Person)`
- Run it: `structured_model.invoke("...")`
- Catch a mismatch: wrap the call in `try/except Exception as e:` and print `type(e)` .

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

`.with_structured_output(YourModel)` is a method on a chat model that returns a *new* runnable — one whose output is already parsed and validated into `YourModel`, instead of raw text. This is different from manually chaining a `PydanticOutputParser` after the model:

`.with_structured_output(...)` uses the provider's own structured-output feature under the hood when it's available, which is more reliable than asking the model nicely in the prompt and hoping the reply parses.

The key thing to observe: when the model's answer genuinely can't be coerced into your Pydantic shape, the parsing step raises a validation error — and that happens *after* the model has already replied, which is different from Doc04's `response_format`, where the shape is enforced *during* generation.

You can still put this inside a full LCEL chain — `prompt | ChatOpenAI().with_structured_output(Person)` — the parsing step is now built into the model piece itself, so you don't need a separate parser step after it.

Difference between Basic and Intermediate: Basic detects a mismatch and prints that it happened. Intermediate catches the specific `ValidationError` class and explains why the failure happens *after* the model replies, unlike Doc04's structured output.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 2 — The plan, and almost the whole thing

Basic Version

```
define a shape called Person with fields: name (text), age (number)

build a model that always answers in the Person shape

test 1: ask it about someone with a clear name and age
      -> print the result, confirm it's a real Person, not text

test 2: ask it something that doesn't have a name and age in it
      -> wrap in try/except, print what kind of error happens
```

Here is almost the whole thing:

```
# structured_output_practice.py – Intermediate section
from pydantic import BaseModel
from langchain_openai import ChatOpenAI

class Person(BaseModel):
    name: str
    age: int

structured_model = ChatOpenAI().with_structured_output(Person)
```

Expected output if you run just this: nothing yet — add the 2 test calls (one printed directly, one inside `try/except`) below it.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

```
class Person(BaseModel):
    name: str
    age: int

structured_model = ChatOpenAI(model="...", temperature=0).with_structured_output(

good_result = structured_model.invoke("Extract: Maria is 34 years old.")
print(good_result)                                # a real Person object

try:
    bad_result = structured_model.invoke("Extract: the sky is blue.")
except Exception as e:
    print(f"Caught {type(e).__name__}: {e}")
```

Notice you're printing `type(e).__name__`, not just the message — knowing the *exact* exception class is what tells your production code which `except` clause needs to catch it. Write this out fully, run it, write down the exact class name you actually get, then compare against the [Solution](#).

Difference between Basic and Intermediate: Basic builds one structured model and observes the raised error on a mismatch. Intermediate catches the specific `ValidationError` class, with a broader fallback after it — the same "catch the specific error you know how to handle" rule from Doc01.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Full solution: [Show me the solution](#)